

Co-Pilot

AI

Scientist

v3：面向协作式自动科研的人类引导假设演化与程序化搜索

作者：何石，新加坡国立大学计算机学院

摘要

自动科研智能体已经开始把创意生成、实验执行、benchmark

评估和论文写作连接成闭环。然而，完全自动化的科研流水线仍然难以处理一些关键节点：选择什么问题值得做、如何判断证据强弱、如何解释失败结果、以及哪些结论可以负责任地写进论文。本文提出

Co-Pilot

AI

Scientist

v3，一种面向人类科研工作者的协作式自动科研架构。它在

AI

Scientist-v2

的基础上加入结构化人类参与节点，覆盖假设形成、分支选择、评估器设计和最终论证审计。同时，该系统综合

AI

Co-Scientist

的假设生成与辩论机制、AI

Scientist-v2

的实验与论文自动化机制，以及

AlphaEvolve

的可自动评估子问题程序搜索机制。本文定义一个可复现评估协议，用于比较完全自动、局部人类参与和完整

co-pilot

版本在小型自动科研任务上的表现。核心假设是：如果把有限的人类注意力放在高杠杆节点，系统可以在保留

agentic

search

可扩展性的同时，提高论文的新颖性、严谨性和证据对齐程度。

1.

引言

自动科研的下一步，不应该只是“无人参与的论文工厂”。科学研究不仅是可执行步骤的串联，也包括选择有价值问题、识别弱证据、重新解释失败、决定哪些主张值得提出。AI

Scientist-v2

展示了

agentic

tree

search

如何把假设变成实验和论文；AI

Co-Scientist

展示了多智能体系统如何生成、讨论并演化科学假设；AlphaEvolve

展示了在存在自动评估函数时，LLM

引导的程序进化可以产生强发现。这些系统共同指向一种新架构：让机器进行大规模搜索，同时让人类在最需要判断力的节点介入。

本文提出

Co-Pilot

AI

Scientist

v3。目标不是用随意审批拖慢自动流水线，而是识别哪些人类参与节点真正能改变研究轨迹。我们关注五类节点：创意假设节点、benchmark/评估器节点、树搜索分支节点、程序化搜索升级节点，以及最终论文主张审计节点。

2.

相关工作

AI

Co-Scientist

将科学发现建模为假设生成、讨论和演化，优势在于研究早期的想象力和证据组织。AI

Scientist-v2

通过

agentic

tree

search

将候选想法转化为可运行实验和论文。AlphaEvolve

使用自动评估驱动代码进化，适合机器可评分问题。FunSearch

是

LLM

引导程序搜索用于数学发现的早期代表。Coscientist

则展示了

LLM

agent

如何连接化学工具和实验自动化。

本文把这些系统视为互补层，而不是彼此替代的端到端方案。

3.

方法

Co-Pilot

AI

Scientist

v3

包含四个循环。

第一是假设循环：系统生成候选研究方向，批评这些方向，将其连接到文献证据，并让人类科学家选择或改写最有潜力的方向。

第二是实验循环：系统把被选中的假设转化为

benchmark

任务、baseline、消融实验和可执行脚本。昂贵实验开始前，人类科学家可以审查并批准评估器。

第三是搜索循环：系统运行

AI

Scientist-v2

风格的实验树搜索。在预设检查点，系统总结当前分支前沿，并让人类科学家决定预算继续投向哪些方向。

第四是程序化优化循环：系统把机器可评分的子问题交给

AlphaEvolve

风格的引擎。由于官方

AlphaEvolve

没有开源核心系统，本文的可复现实验使用

OpenEvolve

作为实际实现底座。OpenEvolve

提供自定义

evaluator、OpenAI-compatible

模型路由、MAP-Elites

质量多样性搜索、岛屿种群和可复现随机种子。该循环进化代码、保存候选程序，并把通过验证的改进返回主科研流程。

每一次人类介入都被记录为结构化数据，包括决策类型、候选选项、理由、影响到的产物和后续结果。这样，人类参与不是隐藏的旁路，而是可复现记录的一部分。

4.

Benchmark

选择与评估计划

我们计划比较六种系统版本：完全自动
baseline、仅创意节点介入、仅分支节点介入、仅评估器节点介入、仅论文主张审计介入，以及完整
co-pilot
v3。Benchmark
不应只局限于
FML-bench，而应根据论文主张分层选择。对于
AI
Scientist-v2
风格的实验搜索，FML-bench
是近期最合适的载体，因为它已经能在
Ubuntu
环境中跑通，并且能产生分支级日志。对于机器可评分的算法子问题，我们使用
OpenEvolve-controlled
tasks，例如函数最小化和
0/1
knapsack
启发式搜索。对于
FML-bench
之外的更广泛证据，本文已加入一个
MLAgentBench
vectorization
小实验来测试端到端
ML
实验能力；下一步应加入
ScienceAgentBench
来测试更接近科学发现的数据驱动任务。更高成本的
stretch
benchmark
包括
MLE-bench
Lite、PaperBench
和
AIRS-Bench：前者测试
Kaggle
风格
ML
engineering，PaperBench
测试从论文到代码复现与层级
rubric
评分，AIRS-Bench
则更接近完整
ML
research
lifecycle。

指标包括任务分数、论文质量、主张支持率、搜索效率、人类注意力成本和假设多样性。对于程序化搜索模块，关键消融是：在相同 evaluator 和迭代预算下，比较 OpenEvolve 与直接重复 LLM 代码编辑。因此，FML-bench 应被理解为当前证据来源之一，而不是整个项目的完整 benchmark 定义。

###

4.1 初步程序化搜索

smoke test

作为可行性检查，我们已经在 SSH 控制的 Ubuntu 主机上运行了 OpenEvolve 0.2.27，并通过 OpenAI-compatible API 调用 DeepSeek。任务是一个小型函数最小化 evaluator，初始程序为随机搜索。一次 OpenEvolve 迭代后，系统将 evaluator 分数从 0.0345 提高到 0.0378，并保存了一个模拟退火风格的 evolved program。这个结果只是 smoke test：它证明远程 OpenEvolve 路径、evaluator 加载、模型路由、checkpoint 和 artifact 捕获可以跑通，但还不能证明本文核心主张，即 human-gated research 能提升最终论文质量。

我们还在相同
DeepSeek
模型、相同初始程序和相同
evaluator
下运行了
direct
one-shot
LLM-edit
baseline。该直接编辑基线得分为
0.038021，略高于一次迭代
OpenEvolve
的
0.037816，也略高于五次迭代
OpenEvolve
的
0.038007。这个边界结果支持
Co-Pilot
AI
Scientist
v3
的一个核心设计：程序化搜索不应该默认总是启动，而应该由明确的升级
gate
控制。对于极小预算或简单局部改写，直接编辑可能已经足够；当搜索空间更丰富、预算更大时，OpenEvol
ve
风格的种群搜索才更可能发挥价值。

为了测试后一种情况，我们进一步加入了一个更复杂的
0/1
knapsack
启发式任务。初始
value/weight
贪心程序在
18
个确定性实例上的平均最优比为
0.991519。direct
LLM
rewrite
将分数提升到
0.995270。五次迭代
OpenEvolve
进一步提升到
0.999439，且没有非法实例。进化出的程序加入了局部改进：移除一个或两个已选物品后，再用贪心方式重
新填充容量。这个结果为升级
gate
提供了正例：当子问题具有更丰富的组合结构，并且存在自动
evaluator
时，OpenEvolve-style
search
确实可能优于直接编辑。

###

4.2

AI

Scientist-v2

分支

gate

回放分析

我们还对已有的两个
AI
Scientist-v2/FML-bench
smoke
run
做了回放分析。在
Causality_causalm1
任务中，第一个
draft
就取得了四步运行中的最佳验证
MAE：0.598943，而原始
baseline
为
1.296259；第二个
draft
和后续
refinement
都没有超过它。如果在两个
draft
之后加入
branch
gate，人类会保留第一条分支，并避免至少一次低价值后续尝试。在
Fairness_fairlearn
任务中，第一个
draft
反而让公平性指标比
baseline
更差：0.316467
对比
baseline
0.186632；第二次尝试因为
Fairlearn
兼容性问题失败，后续尝试也要么更差、要么出错。此时
human
branch
gate
应暂停该分支，并把系统路由到
evaluator/兼容性修复或新的公平策略。这个回放证据还不能替代在线
human-gated
benchmark，但它说明
AI
Scientist-v2
的日志轨迹中已经存在适合人类
co-pilot
介入的可操作检查点。

###

4.3

在线

AI

Scientist-v2

分支

gate

小实验

为了从回放证据推进到在线证据，我们在

Ubuntu

主机上新跑了一次

FML-bench

小实验。任务仍为

Causality_causalm1。我们把

AI

Scientist-v2

设置为两步预算、两个

draft

idea、两个模拟并行

worker，并用临时

agent

配置把全部预算放到第一阶段，从而强制产生两个可比较的

draft

分支。第一个

draft

的验证

MAE

为

1.149610，第二个

draft

的验证

MAE

为

0.621262；该任务中

MAE

越低越好。因此，结构化

branch

gate

选择第二个

draft，并剪枝第一个

draft。benchmark

自动对最佳验证分支做最终测试，得到

test

MAE

0.640451。

这个在线小实验说明，本文提出的

branch

gate

可以插入真实

AI

Scientist-v2

决策前沿，并基于日志指标和代码快照做出清晰的预算分配决策。它还不能证明

human

gating

优于完全自动

AI

Scientist-v2：该实验尚未从被选中的分支继续

resume

进行后续

human-gated

search，而且其测试分数略弱于之前的四步

autonomous

smoke

run。因此，我们把它定位为在线人类介入可行性证据，而不是系统级优势的最终证据。

随后，我们实现了一个最小
selected-branch
continuation
runner。该
runner
会把人类
gate
选中的代码快照临时写回官方
FML-bench
任务模板，从该状态启动一个短预算
AI
Scientist-v2
continuation
run，并在结束后恢复模板文件。从被选中的第二个
draft
出发，继续两步后验证
MAE
达到
0.401240， test
MAE
达到
0.402170。这个结果优于
live
two-draft
gate
run
的
test
MAE
0.640451，也优于早期四步
autonomous
smoke
run
的
test
MAE
0.617719。这个比较仍然是初步的：当前
continuation
是通过
snapshot
seeding
实现的，还没有保存原始内存中的
tree
对象；同时它只覆盖一个小任务和一个
seed。因此，我们只把它作为“branch
gate
->
selected
snapshot
->
additional
AI
Scientist-v2
budget”这条路径可以执行的证据，而不是最终统计结论。

4.4
非
FML
benchmark
与程序搜索小实验
根据上面的
benchmark
选择原则，我们还把
MLAgentBench
作为非
FML
评估来源。我们在
Ubuntu
主机上克隆
MLAgentBench，并先运行其轻量
vectorization
任务，使用内置
Agent
baseline。该
baseline
只执行
starter
train.py
并提交结果。官方
baseline
成功完成，final
score
为
3.172504
秒，total
benchmark
time
为
3.365531
秒，且没有错误标记。

随后，我们把同一个任务封装成更严格的本地
evaluator：在接受运行时间之前，evaluator
会用一个小型确定性输入，把候选
Conv2DLayer.forward
的输出和
nested-loop
reference
做数值比对。在这个受控
evaluator
下，starter
program
的
median
runtime
为
3.261186
秒。一次
direct
DeepSeek
deepseek-chat
rewrite
生成了看似合理的向量化代码，但因为数值不一致没有通过
correctness
gate。相反，使用同一
DeepSeek
模型的三轮
OpenEvolve-style
run
在第
1
轮找到了正确候选，median
runtime
为
0.051882
秒，相比受控
starter
program
约快
62.86
倍。这个结果支持一个较窄但重要的结论：program-search-escalation
节点可以在
FML-bench
之外的机器可评分子问题上发挥作用。但它还不能证明完整
co-pilot
架构能提升整篇论文质量。

为了检查稳健性，我们又用两个额外

random

seed

重复同样的三轮

OpenEvolve

设置。seed

7

没有找到加速程序，最终保留近似

starter

的程序，median

runtime

为

2.984610

秒；seed

123

找到了正确向量化程序，median

runtime

为

0.031783

秒。在

seed

42、7、123

三次运行中，系统有

2

次找到正确加速程序，全

seed

的

best

runtime

中位数为

0.051882

秒。这增强了“程序搜索模块可以找到有效代码变换”的证据，同时也说明在极小预算下存在

seed

sensitivity。

###

4.5

主张审计

在完成这些
pilot
实验后，我们做了一次
claim-evidence
audit。通过
Monica
路由的
gpt-4o-mini
审稿式检查认为：本文的架构贡献是合理的，但当前实验证据还不足以支持“人类
gate
提高论文质量”或“完整
co-pilot
系统优于
autonomous
AI
Scientist-v2”这类宽泛结论。因此，本文把这些表述保留为
evaluation
protocol
要检验的假设，而不是已经证明的结论。当前
audit
只支持较窄的实证主张：OpenEvolve-style
search
可以在部分机器可评分子问题上有效，但在极小预算下存在
seed
sensitivity；branch
gate
可以插入
AI
Scientist-v2
风格日志轨迹；selected-branch
continuation
在单任务上有积极结果，但仍需要同预算、多
seed
的验证。

5.

当前贡献与尚未证明的主张

本文当前贡献包括：

1. 一个面向协作式自动科研的模块化架构。
2. 一个用于科研
agent
的人类参与节点形式化
schema。
3. 一个评估“人类注意力是否以及应该放在哪里”的实验协议。

4.
远程
OpenEvolve
与
FML-bench
小实验，证明
AlphaEvolve-style
子问题模块和
AI
Scientist-v2
分支
gate
模块可以在
Ubuntu
主机上运行。
5.
一个非
FML
的
MLAgentBench
程序搜索小实验，用于扩展
FML-bench
之外的
benchmark
覆盖面。
6.
一个可在
Codex
中复用的
workflow
skill。
7.
中英文论文、使用文档和主张审计
artifact，便于复现和传播。
当前证据还不能证明人类
gate
能提升论文质量，也不能证明完整
co-pilot
系统优于
autonomous
AI
Scientist-v2。这些仍是下一阶段
benchmark
要验证的目标主张。
6.
局限性

本文不预设人类参与一定有效。人类

gate

可能引入偏见、降低搜索速度、压缩探索多样性。程序化搜索可能只优化局部指标，却不能提高论文层面的科学贡献；在极小预算下，它也未必优于直接

LLM

编辑。专家论文评分成本较高，而且不同评审可能存在分歧。因此，第一版实验应保持窄主张，并在

gate

无法改善结果时如实报告负结果。当前

selected-branch

continuation

结果很有希望，但还不是统计受控

benchmark；更强主张需要更多任务、更多随机种子、匹配的

autonomous

budget，以及独立论文质量评审。

7.

结论

Co-Pilot

AI

Scientist

v3

将自动科学发现重新定义为协作式搜索。该系统保留自动

agent

的大规模搜索能力，同时给人类科学家提供明确、可记录、可实验检验的影响节点。如果未来

matched

benchmark

支持中心假设，该架构可能通过把人类注意力放在边际价值最高的位置来提高科研质量，而不是把人类排除在科学之外。当前论文应被理解为一个带

pilot

evidence

的可复现系统

proposal，而不是最终优越性证明。